Module 3 — Advanced SQL for product-company screens

Learning outcomes

Use window functions, date logic, ranking, deduplication, rolling metrics, gap detection and execution plans.

1. Window functions

A window function calculates across related rows without collapsing them like GROUP BY.

• `ROW_NUMBER`: unique sequence.

• `RANK`: ties share rank; gaps follow.

• `DENSE_RANK`: ties share rank; no gaps.

• `LAG`/`LEAD`: previous/next row value.

• Aggregate windows: running sum, moving average, cumulative count.

2. Latest record per entity

Use `ROW_NUMBER() OVER (PARTITION BY key ORDER BY timestamp DESC, stable_tie_breaker DESC)`. Never rely on arbitrary ordering when timestamps tie.

3. Date boundaries

Prefer half-open intervals: `>= start AND < next_period_start`. This avoids missing timestamps later on the final day. Be explicit about timezone and whether a timestamp is stored in UTC.

4. Missing calendar periods

Grouping only returns periods present in the data. For a complete monthly series, join to a calendar/date dimension and fill missing activity with zero where that is semantically valid.

5. Rolling windows

A `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` frame means seven rows, not necessarily seven calendar days. Create a daily date spine before using it as a seven-day metric.

6. Deduplication

Define the business key, version ordering and winner. Preserve raw data. A duplicate can mean repeated delivery of the same event, not two legitimate identical purchases.

7. Query performance

• Inspect `EXPLAIN` and, where safe, `EXPLAIN ANALYZE`.

• Compare estimated and actual row counts.

• Check scan type, join algorithm, sort, aggregate and spills.

• Use selective predicates and project only needed columns.

• Keep indexed filter columns sargable.

• Indexes have write/storage costs.

• Statistics and data distribution influence plans.

Worked examples

A. Rank stores by revenue in each city

```
WITH store_sales AS (
  SELECT city, store_id, SUM(net_sales) AS revenue
  FROM daily_store_sales
  GROUP BY city, store_id
)
SELECT *,
```

```
       DENSE_RANK() OVER (
         PARTITION BY city ORDER BY revenue DESC
       ) AS city_rank
FROM store_sales;
```

B. Detect repeat purchases within 30 days

```
WITH ordered AS (
  SELECT customer_id, order_id, order_ts,
         LAG(order_ts) OVER (
           PARTITION BY customer_id ORDER BY order_ts, order_id
         ) AS previous_order_ts
  FROM orders
)
SELECT *,
       (order_ts - previous_order_ts) <= INTERVAL '30 days' AS within_30_days
FROM ordered
WHERE previous_order_ts IS NOT NULL;
```

C. Find gaps in a sequence of invoice numbers

```
WITH nums AS (
  SELECT invoice_no,
         LAG(invoice_no) OVER (ORDER BY invoice_no) AS prev_no
  FROM invoices
)
SELECT prev_no + 1 AS gap_start, invoice_no - 1 AS gap_end
FROM nums
WHERE prev_no IS NOT NULL
  AND invoice_no > prev_no + 1;
```

Guided exercises with solutions

1. Return the third most recent login per user.

```
WITH x AS (
 SELECT *, ROW_NUMBER() OVER (
   PARTITION BY user_id ORDER BY login_ts DESC, login_id DESC
 ) AS rn
 FROM logins
)
SELECT * FROM x WHERE rn = 3;
```

2. Find products whose revenue is above their category average.

```
WITH p AS (
 SELECT category, product_id, SUM(revenue) AS revenue
 FROM product_sales GROUP BY category, product_id
)
SELECT *
FROM (
 SELECT p.*,
        AVG(revenue) OVER (PARTITION BY category) AS category_avg
 FROM p
) x
WHERE revenue > category_avg;
```

3. Calculate cumulative daily active users from a daily summary table.

```
SELECT activity_date, active_users,
       SUM(active_users) OVER (
         ORDER BY activity_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS cumulative_daily_user_days
FROM daily_activity;
```

This is cumulative user-days, not cumulative distinct users. A user can appear on multiple days.

4. Return users active on at least 5 distinct days in a 7-day period. Explain that this requires a per-user rolling date window, not simply a global daily count. One approach:

```
SELECT DISTINCT a.user_id
FROM activity a
WHERE (
  SELECT COUNT(DISTINCT b.activity_date)
  FROM activity b
  WHERE b.user_id = a.user_id
    AND b.activity_date BETWEEN a.activity_date - 6 AND a.activity_date
) >= 5;
```

5. Find each customer's longest gap between orders.

```
WITH x AS (
 SELECT customer_id, order_ts,
        LAG(order_ts) OVER (
          PARTITION BY customer_id ORDER BY order_ts
        ) AS prev_ts
 FROM orders
)
SELECT customer_id, MAX(order_ts - prev_ts) AS longest_gap
FROM x
WHERE prev_ts IS NOT NULL
GROUP BY customer_id;
```

Independent interview exercises

6. Return all tied employees at the highest salary in each department.

7. Find the first purchase date after signup.

8. Calculate 90th percentile order value by country.

9. Find users who had activity on three consecutive calendar days.

10. Explain why `COUNT(DISTINCT user_id)` across all events is not the same as summing daily active users.

Solutions

```
-- 6
SELECT *
FROM (
 SELECT e.*,
        RANK() OVER (
          PARTITION BY department_id ORDER BY salary DESC
        ) AS rnk
 FROM employees e
) x
WHERE rnk = 1;

-- 7
SELECT c.customer_id, MIN(o.order_ts) AS first_purchase
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'COMPLETED'
  AND o.order_ts >= c.signup_ts
GROUP BY c.customer_id;

-- 8 (PostgreSQL)
SELECT country,
       PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY order_value) AS p90
FROM orders
GROUP BY country;

-- 9 (PostgreSQL date arithmetic)
WITH d AS (
 SELECT DISTINCT user_id, activity_date
 FROM user_activity
), x AS (
 SELECT user_id, activity_date,
        LAG(activity_date, 1) OVER (PARTITION BY user_id ORDER BY activity_date) AS d1,
        LAG(activity_date, 2) OVER (PARTITION BY user_id ORDER BY activity_date) AS d2
 FROM d
)
SELECT DISTINCT user_id
FROM x
WHERE activity_date = d1 + 1
  AND d1 = d2 + 1;

-- 10
-- Daily active user counts overlap; the same person can be active on multiple days.
-- Sum daily DAU = user-days, not unique users over the entire period.
```

Mock test — advanced SQL (30 minutes, 30 marks)

• 8 marks: top two sellers per region including ties.

• 8 marks: latest valid status per shipment.

• 7 marks: week-over-week revenue with missing weeks.

• 7 marks: explain and correct an inflated aggregate after a one-to-many join.

Mark for correct grain, deterministic ordering, missing-period logic and clear explanation.

External practice for this module

1. LeetCode SQL 50 — complete the Advanced Select and Joins and Subqueries sections. Re-solve all Medium questions you miss.

2. LeetCode Advanced SQL 50 — use as optional extension if available to the account; do not pay solely for this module.

3. PostgreSQL Exercises — window functions — complete the window-function and date-related exercises.

4. DataLemur — solve 10 interview questions involving ranking, latest records, rolling metrics, dates or consecutive activity.

5. Use The Index, Luke — study index fundamentals and query performance; then inspect `EXPLAIN` plans in a local PostgreSQL database.

Exit requirement: solve 8 window/date problems in 60 minutes. Include latest-row-per-key, top-N-per-group, LAG/LEAD, rolling period and missing calendar periods. Explain why `ROWS` is not automatically equivalent to a calendar-day window.